

API Design

Backend Engineering — Topic 10 · Study Notes

Contents

1. REST API Design

- 1.1 What REST actually is
- 1.2 Richardson Maturity Model
- 1.3 Resource naming rules
- 1.4 HTTP methods — semantics that matter
- 1.5 Status codes worth memorising
- 1.6 Response shape: envelope vs raw
- 1.7 Error format
- 1.8 Other essentials
- 1.9 Laravel sketch

2. Pagination

- 2.1 Offset / Limit (page-number) pagination
- 2.2 Cursor / Keyset (seek) pagination
- 2.3 Response shape
- 2.4 Practical rules
- 2.5 Laravel

3. Filtering and Sorting

- 3.1 Filtering conventions
- 3.2 Sorting
- 3.3 Security — the part people get wrong
- 3.4 Related concepts
- 3.5 Laravel

4. API Versioning

- 4.1 What actually needs a version
- 4.2 Strategies
- 4.3 Deprecation and sunseting
- 4.4 Structuring the code

5. Rate Limiting

- 5.1 Why
- 5.2 Algorithms
- 5.3 Where and by what key
- 5.4 The response
- 5.5 Distributed implementation
- 5.6 Laravel

6. GraphQL Basics

- 6.1 The problem it solves
- 6.2 The schema
- 6.3 Resolvers
- 6.4 N+1 — the defining GraphQL problem
- 6.5 Transport, errors, caching
- 6.6 Pagination — Relay connections
- 6.7 Security — where GraphQL gets dangerous
- 6.8 REST vs GraphQL

Practice Plan

1. REST API Design

1.1 What REST actually is

REST (Representational State Transfer) is an *architectural style*, not a protocol. Roy Fielding defined six constraints:

Constraint	Meaning
Client-Server	UI and data storage are separate concerns; they evolve independently
Stateless	Every request carries everything the server needs. No server-side session between requests
Cacheable	Responses declare whether they can be cached (<code>Cache-Control</code> , <code>ETag</code>)
Uniform Interface	Resources identified by URI, manipulated via representations, self-descriptive messages, HATEOAS
Layered System	Client can't tell if it's talking to the origin server or a proxy/LB/CDN
Code-on-Demand (optional)	Server can ship executable code to client (rarely used)

Stateless is the one people break most. If your API stores “current step of wizard” in a PHP session, it isn't REST — and it won't scale horizontally without sticky sessions or shared session storage.

1.2 Richardson Maturity Model

A ladder for “how RESTful is this API?”

- **Level 0** — Single endpoint, everything is POST. (SOAP/RPC style: `POST /api` with `{"action":"getUser"}`)
- **Level 1** — Resources. `/users` , `/orders` exist as distinct URIs, but still all POST.
- **Level 2** — HTTP verbs + status codes used properly. **This is where 95% of real-world “REST” APIs live, and that's fine.**
- **Level 3** — HATEOAS. Responses include links telling the client what it can do next.

```
{
  "id": "a3f9k",
  "status": "pending",
  "_links": {
    "self": { "href": "/v1/bookings/a3f9k" },
    "cancel": { "href": "/v1/bookings/a3f9k/cancel", "method": "POST" },
    "pay": { "href": "/v1/bookings/a3f9k/payment", "method": "POST" }
  }
}
```

HATEOAS is genuinely useful when the *available actions* depend on state — e.g. a booking can only be cancelled before departure. Instead of the mobile app reimplementing that rule, the server tells it.

1.3 Resource naming rules

```
✓ GET    /v1/buses
✓ GET    /v1/buses/{id}
✓ GET    /v1/buses/{id}/seats
✓ POST   /v1/bookings
✓ DELETE /v1/bookings/{id}
```

```
✗ GET    /v1/getBuses           (verb in URI – the HTTP method IS the verb)
✗ POST   /v1/bus/create
✗ GET    /v1/Bus_List
```

Conventions:

- **Nouns, not verbs.** The method supplies the verb.
- **Plural collections.** `/buses` not `/bus`. Be consistent — mixing is worse than either choice.
- **lowercase-kebab-case** for multi-word: `/trip-schedules`, not `/tripSchedules` or `/trip_schedules`.
- **Nest one level deep, max two.** `/buses/{id}/seats` is fine. `/companies/{a}/buses/{b}/trips/{c}/seats/{d}` is not — expose `/seats?trip_id={c}` instead.
- **Query strings are for filtering the collection, not identifying a resource.** `/buses/12` identifies; `/buses?type=ac`

filters.

Actions that aren't CRUD. Real systems have “cancel”, “refund”, “publish”, “resend OTP”. Two acceptable patterns:

1. Sub-resource as a noun: `POST /bookings/{id}/cancellation`
2. Controller-style verb endpoint: `POST /bookings/{id}/cancel`

Option 2 is pragmatic and widely used (Stripe, GitHub both do it). Don't twist yourself into knots for purity.

1.4 HTTP methods — semantics that matter

Method	Purpose	Safe?	Idempotent?	Body?
GET	Read a resource	✓	✓	No
HEAD	Like GET, headers only	✓	✓	No
POST	Create / non-idempotent action	✗	✗	Yes
PUT	Full replace (or create at known URI)	✗	✓	Yes
PATCH	Partial update	✗	✗*	Yes
DELETE	Remove	✗	✓	Usually no
OPTIONS	Discover allowed methods / CORS preflight	✓	✓	No

- **Safe** = no observable state change on the server. Never let `GET /orders/{id}/delete` exist. Crawlers and prefetchers will find it.
- **Idempotent** = doing it N times has the same effect as doing it once. `DELETE /bookings/5` twice: first returns 204, second returns 404 (or 204). Server state is the same either way — that's idempotent.
- *PATCH is idempotent only if your patch document is absolute (`{"seats": 3}`). It is **not** if it's relative (`{"seats": "+1"}`).

PUT vs PATCH: PUT means “here is the complete new representation; anything I omit should be nulled/defaulted.” PATCH means “apply just these changes.” Most APIs accept PATCH semantics but expose it under PUT, which causes subtle bugs. Pick one and document it.

1.5 Status codes worth memorising

2xx — Success

- `200 OK` — general success with a body
- `201 Created` — resource created; include a `Location: /v1/bookings/a3f9k` header
- `202 Accepted` — queued for async processing; body should point at a status URL
- `204 No Content` — success, deliberately empty body (typical for DELETE)

3xx — Redirect

- `301 Moved Permanently` / `308` (preserves method)
- `304 Not Modified` — conditional GET hit the cache (`If-None-Match` matched your `ETag`)

4xx — Client's fault

- `400 Bad Request` — malformed syntax, unparseable JSON
- `401 Unauthorized` — actually means *unauthenticated*. No/invalid credentials.
- `403 Forbidden` — authenticated, but not allowed
- `404 Not Found` — no such resource (also used to hide existence from unauthorised users)
- `405 Method Not Allowed` — must include an `Allow` header
- `409 Conflict` — version conflict, duplicate unique key, seat already booked
- `410 Gone` — used to exist, permanently removed (good for retired API versions)
- `422 Unprocessable Entity` — syntactically valid but semantically wrong — **validation errors go here**
- `429 Too Many Requests` — rate limited

5xx — Your fault

- **500 Internal Server Error** — unhandled exception. Never leak a stack trace.
- **502 Bad Gateway** — upstream returned garbage
- **503 Service Unavailable** — down/overloaded; include **Retry-After**
- **504 Gateway Timeout** — upstream too slow

400 vs 422 is the classic interview question. 400 = “I can’t even parse this.” 422 = “I parsed it fine, but **email** is already taken.”

1.6 Response shape: envelope vs raw

Your assessment API uses an envelope:

```
{
  "flag": true,
  "msg": "Buses fetched successfully",
  "data": { "buses": [ ... ] },
  "response_code": 200
}
```

Envelope pros: uniform parsing on the client, room for **meta/msg** alongside data, survives transports that mangle status codes (JSONP, some corporate proxies). **Envelope cons:** duplicates information HTTP already carries, adds a layer of unwrapping, and tooling (OpenAPI generators, HTTP caches) understands status codes but not your **flag** field.

The cardinal sin is returning 200 OK with {"flag": false} for an error. Then monitoring, load balancers, and retry logic all think everything is healthy. If you use an envelope, still set the real HTTP status.

1.7 Error format

Be consistent and machine-readable. RFC 7807 (**application/problem+json**) is the standard:

```
{
  "type": "https://api.example.com/errors/seat-unavailable",
  "title": "Seat is no longer available",
  "status": 409,
  "detail": "Seat B4 on trip TRP-8821 was booked 3 seconds ago.",
  "instance": "/v1/bookings",
  "seat_no": "B4"
}
```

Validation errors need a field map:

```
{
  "status": 422,
  "title": "Validation failed",
  "errors": {
    "email": ["The email has already been taken."],
    "seats": ["The seats field must be an array."]
  }
}
```

Rules: use a **stable machine code** (**SEAT_UNAVAILABLE**) that clients branch on, plus a human message they can display. Never make clients regex your prose. Include a **request_id** so support can trace it in logs.

1.8 Other essentials

- **Auth** — Bearer tokens in **Authorization: Bearer <jwt>**. Never in query strings (they land in access logs and browser history).
- **Idempotency keys** — for POST endpoints that move money. Client sends **Idempotency-Key: <uuid>**; server stores key → response for 24h and replays it on retry. Essential for bKash/SSLCOMMERZ-style flows where the network can drop after the charge succeeds.
- **Optimistic concurrency** — return an **ETag**; require **If-Match** on updates; return **412 Precondition Failed** on mismatch. Prevents lost updates.
- **Timestamps** — always ISO 8601 with timezone (**2026-08-05T14:30:00+06:00**) or UTC with **Z**. Never **05/08/2026**.
- **Money** — integer minor units (**"amount": 125000, "currency": "BDT"**) or decimal strings. Never floats.
- **Nulls** — decide whether to omit or send **null**. Be consistent.
- **Compression, CORS, request size limits, Content-Type enforcement.**

1.9 Laravel sketch

```
// routes/api.php
Route::prefix('v1')->group(function () {
    Route::apiResource('buses', BusController::class);
    Route::post('bookings/{booking}/cancel', [BookingController::class, 'cancel']);
});

// StoreBookingRequest - validation → auto 422 with error bag
public function rules(): array {
    return [
        'trip_id' => ['required', 'string', 'exists:trips,hashid'],
        'seats'   => ['required', 'array', 'min:1', 'max:4'],
        'seats.*' => ['string', 'distinct'],
    ];
}

// BusResource - controls the wire format, decoupled from DB columns
public function toArray($request): array {
    return [
        'id'       => $this->hashid,
        'name'     => $this->name,
        'type'     => $this->type,
        'departs_at' => $this->departs_at->toIso8601String(),
        'operator' => new OperatorResource($this->whenLoaded('operator')),
    ];
}
```

Always put an **API Resource** between your Eloquent model and the response. Otherwise a new DB column silently leaks into your public contract.

2. Pagination

Never return an unbounded collection. `GET /bookings` on a table with 4M rows will OOM your worker.

2.1 Offset / Limit (page-number) pagination

`GET /v1/bookings?page=3&per_page=25`

```
SELECT * FROM bookings ORDER BY created_at DESC LIMIT 25 OFFSET 50;
```

Pros: trivial to implement, users can jump to page 47, total count and page count available.

Cons — two serious ones:

1. **Deep pagination is O(offset).** `OFFSET 500000` forces the DB to read and discard 500,000 rows. Page 1 is 2ms; page 20,000 is 8 seconds.
2. **Result drift.** Between fetching page 1 and page 2, someone inserts a new booking at the top. Every row shifts down one — the user sees one record twice and never sees another.

Also, `SELECT COUNT(*)` for the total is itself expensive on large tables (in InnoDB it's a full index scan).

Use it for: admin panels, dashboards, anything where a human clicks through a few pages of a modest dataset.

2.2 Cursor / Keyset (seek) pagination

Instead of “skip 500,000 rows”, say “give me rows *after this one*.”

`GET /v1/bookings?limit=25`

`GET /v1/bookings?limit=25&after=eyJpZCI6OTkyMSwiY3JlYXRlZF9hdCI6IjIwMjYtMDgtMDVUMTA6MDA6MDBaIn0`

```
SELECT * FROM bookings
WHERE (created_at, id) < ('2026-08-05 10:00:00', 9921)
ORDER BY created_at DESC, id DESC
LIMIT 25;
```

Key points:

- **Sort key must be unique**, or you'll skip/duplicate rows at page boundaries. `created_at` alone isn't unique — two bookings in the same second collide. So use a **composite key with a tie-breaker**: `(created_at, id)`.
- The **cursor** is usually the sort-key values, base64-encoded JSON. Encoding signals “opaque, don't parse this” so you can change the internals later. Sign or encrypt it if it might leak internal IDs.
- **Row-value comparison** `(a, b) < (x, y)` is supported by MySQL and Postgres and uses the composite index efficiently. If your DB doesn't support it, expand manually: `a < x OR (a = x AND b < y)`.
- Requires an index on `(created_at, id)`.

Pros: constant-time regardless of depth; stable under concurrent inserts; ideal for infinite scroll and API consumers syncing data.

Cons: no “jump to page 47”; no total count (or an expensive separate query); sorting is constrained to the indexed key.

2.3 Response shape

```
{
  "data": [ ... ],
  "meta": {
    "per_page": 25,
    "current_page": 3,
    "last_page": 12,
    "total": 287
  },
  "links": {
    "first": "/v1/bookings?page=1",
    "prev": "/v1/bookings?page=2",
    "next": "/v1/bookings?page=4",
    "last": "/v1/bookings?page=12"
  }
}
```

Cursor version:

```
{
  "data": [ ... ],
  "meta": { "per_page": 25, "has_more": true },
  "links": { "next": "/v1/bookings?limit=25&after=eyJ..." }
}
```

Giving clients a ready-made `next` URL means they never construct cursors themselves — you keep the freedom to change the scheme.

2.4 Practical rules

- **Cap `per_page`.** Accept the client's value but clamp it: `min($request->per_page ?? 25, 100)`. Otherwise someone sends `per_page=1000000`.
- **Always define a deterministic sort.** Without `ORDER BY`, "page 2" is meaningless — the DB may return rows in any order.
- **Make total counts optional.** `?with_total=true`. Most clients don't need it and it doubles your query cost.
- Alternative transport: `Link` header (RFC 8288) — GitHub's approach. Keeps the body clean.

2.5 Laravel

```
Bus::query()->paginate(25);           // offset + COUNT(*) → full meta
Bus::query()->simplePaginate(25);       // offset, no COUNT → just has-more
Bus::query()->cursorPaginate(25);      // keyset, uses the ORDER BY columns as cursor
```

`cursorPaginate()` derives the cursor from your `orderBy` clauses — so always add `->orderBy('id')` as a tie-breaker.

Practice: implement cursor pagination by hand (no `cursorPaginate`) on a table of 1M seeded rows, and benchmark it against `paginate()` at page 1, page 100, and page 20,000.

3. Filtering and Sorting

3.1 Filtering conventions

Three common styles — pick one, be consistent:

```
# 1. Flat query params (simplest)
GET /v1/buses?status=active&type=ac&operator_id=17

# 2. Bracketed operators (expressive)
GET /v1/buses?price[gte]=800&price[lte]=1500&departs_at[after]=2026-08-10

# 3. JSON:API filter namespace (avoids collisions with page/sort/include)
GET /v1/buses?filter[status]=active&filter[type]=ac
```

Style 3 is worth the extra typing on a large API: it guarantees a filter named `sort` or `page` never collides with your control params.

Operator vocabulary — define it once, use it everywhere:

Operator	Meaning	Example
<code>eq</code> / bare	equals	<code>status=active</code>
<code>ne</code>	not equal	<code>status[ne]=cancelled</code>
<code>gt</code> <code>gte</code> <code>lt</code> <code>lte</code>	comparison	<code>price[gte]=800</code>
<code>in</code>	one of	<code>status[in]=pending,paid</code>
<code>between</code>	range	<code>price[between]=800,1500</code>
<code>like</code>	partial match	<code>name[like]=green</code>
<code>null</code>	is / is not null	<code>cancelled_at[null]=true</code>

3.2 Sorting

The dominant convention is a comma-separated list with `-` for descending:

```
GET /v1/buses?sort=-departs_at,name
```

```
→ ORDER BY departs_at DESC, name ASC
```

Alternatives: `?sort_by=departs_at&order=desc` (only supports one field), or `?sort=departs_at:desc,name:asc`. The `-` prefix is cleanest and multi-field.

Always append a unique tie-breaker (`, id ASC`) so ordering is deterministic — otherwise pagination breaks in ways that are maddening to debug.

3.3 Security — the part people get wrong

Never pass user input directly into a column name or SQL fragment.

```
// ✗ SQL injection + information disclosure
$query->orderBy($request->sort);
$query->where($request->field, $request->value);

// ✓ Whitelist
const SORTABLE = ['departs_at', 'price', 'name', 'created_at'];

$sort = $request->query('sort', '-departs_at');
foreach (explode(',', $sort) as $field) {
    $dir = str_starts_with($field, '-') ? 'desc' : 'asc';
    $field = ltrim($field, '-');
    if (!in_array($field, self::SORTABLE, true)) {
        abort(422, "Cannot sort by [$field].");
    }
    $query->orderBy($field, $dir);
}
$query->orderBy('id'); // tie-breaker
```

Same for filters. A whitelist also protects against sorting on an unindexed column and taking down your DB.

Also worth doing:

- **Return 422 on unknown filter keys** rather than silently ignoring them. A typo'd `?statu=active` that silently returns everything is a data leak waiting to happen.
- **Cap in list sizes** and `like` complexity. `name[like]=%a%` on 10M rows is a free DoS.

3.4 Related concepts

Sparse fieldsets — let clients ask for fewer fields:

```
GET /v1/buses?fields=id,name,departs_at
```

Includes / eager loading — let clients pull relations, avoiding N+1 round trips:

```
GET /v1/bookings?include=passenger,trip.operator
```

Whitelist these too, cap the depth, and map each to `with()` so you don't create an N+1 *inside* your server while fixing one on the client.

Search vs filter — `?q=green line dhaka` is fuzzy relevance-ranked text search (full-text index, or Meilisearch/Elasticsearch). `?operator=greenline` is an exact structured filter. Don't implement search as a chain of `LIKE %...%` on five columns; it won't use an index.

3.5 Laravel

```
// Manual, explicit
$buses = Bus::query()
->when($r->filled('status'), fn ($q) => $q->where('status', $r->status))
->when($r->filled('type'), fn ($q) => $q->where('type', $r->type))
->when($r->filled('min_price'), fn ($q) => $q->where('price', '>=', $r->min_price))
->when($r->filled('q'), fn ($q) => $q->whereFullText(['name'], $r->q))
->cursorPaginate(25);
```

Or `spatie/laravel-query-builder`, which gives you allowed-filters/sorts/includes with whitelisting built in.

Performance: every filterable column needs an index, and composite index **column order matters** — an index on `(status, departs_at)` serves `WHERE status=? ORDER BY departs_at` but not `WHERE departs_at=?`. Run `EXPLAIN` on your top three filter combinations.

Practice: build `GET /buses` supporting `filter[]`, `sort`, `fields`, `include`, and cursor pagination, all whitelisted, with 422s on unknown keys.

4. API Versioning

4.1 What actually needs a version

Breaking changes (need a new version):

- Removing or renaming a field or endpoint
- Changing a field's type (`"price": 800` → `"price": "800.00"`)
- Adding a required request parameter
- Tightening validation
- Changing the meaning of an existing value
- Changing default sort order or pagination size (subtler, but clients depend on it)

Non-breaking (ship freely):

- Adding a new optional request parameter
- Adding a new field to a response — *provided* clients are told to ignore unknown fields
- Adding a new endpoint
- Adding a new enum value — *only* if clients handle unknowns gracefully. Otherwise it's breaking.

The best versioning strategy is **not needing one**. Additive changes + tolerant readers gets you years.

4.2 Strategies

1. URI path — `GET /v1/buses`

- ✓ Obvious, easy to route, cacheable, testable from a browser, trivially explained
- ✗ Purists object (the resource is the same, only its representation changed); versions the whole API at once
- **This is what most public APIs use. Default to it.**

2. Query parameter — `GET /buses?version=1`

- ✓ Easy default (omit = latest, or omit = v1)
- ✗ Clutters filter params, easy to forget, cache keys get messy

3. Custom header — `X-API-Version: 2`

- ✓ Clean URIs
- ✗ Invisible in logs/browser, hard to test with curl, proxies may strip it

4. Accept header / content negotiation — `Accept: application/vnd.greenline.v2+json`

- ✓ The “most RESTful” answer; lets you version per-resource
- ✗ Verbose, poor tooling support, most teams get it wrong

5. Date-based versioning — `Stripe-Version: 2026-08-05`

- Client pins a date on signup; the server runs a chain of small transformers that downgrade the current response to the client's pinned date.
- ✓ Extremely fine-grained; no v1/v2/v3 code duplication; hundreds of versions maintainable
- ✗ Real engineering investment; only worth it at Stripe-like scale

4.3 Deprecation and sunseting

Having v2 is easy. Killing v1 is the hard part.

1. **Announce** — changelog, email, dashboard banner. Give a concrete date.

2. **Signal in-band** — RFC 8594 headers on every v1 response:

```
Deprecation: Fri, 05 Aug 2026 00:00:00 GMT
Sunset: Wed, 05 Aug 2027 00:00:00 GMT
Link: <https://docs.example.com/migrate-v2>; rel="deprecation"
```

3. **Measure** — log version + client ID per request. You need to know exactly who's still on v1 before you can turn it off.

4. **Brownouts** — before the sunset date, return `410 Gone` for a few hours on a scheduled day. Nothing surfaces forgotten

integrations like a controlled outage.

5. **Sunset** — [410 Gone](#) permanently, with a body pointing at the migration guide.

Publish a support policy up front (“each major version supported ≥ 12 months after its successor ships”). Without one, every deprecation is a negotiation.

4.4 Structuring the code

```
app/Http/
├── Controllers/Api/
│   ├── V1/BusController.php
│   └── V2/BusController.php
├── Resources/
│   ├── V1/BusResource.php
│   └── V2/BusResource.php
└── Requests/
    ├── V1/StoreBookingRequest.php
    └── V2/StoreBookingRequest.php
```

```
Route::prefix('v1')->namespace('Api\V1')->group(base_path('routes/api_v1.php'));
Route::prefix('v2')->namespace('Api\V2')->group(base_path('routes/api_v2.php'));
```

Key principle: version the edge, share the core. Controllers, Requests, and Resources are versioned. Services, domain logic, models, and DB schema are *not* — otherwise you’re maintaining two applications. If v2 needs different business behaviour, that’s a flag or strategy inside the service, not a forked service.

Practice: take one existing endpoint, ship a v2 where a field is renamed and a nested object flattened, keep v1 working against the same service layer, and add [Deprecation](#) / [Sunset](#) headers to v1.

5. Rate Limiting

5.1 Why

- **Abuse** — credential stuffing on `/login`, OTP spam, scraping your fare table
- **Fairness** — one client's runaway retry loop shouldn't starve everyone else
- **Cost control** — each request costs CPU, DB connections, and possibly third-party API fees
- **Stability** — rate limiting is a load-shedding mechanism; it's how you fail gracefully instead of falling over

Distinguish three related things:

- **Rate limit** — requests per unit time (100/min)
- **Quota** — total allowance per billing period (1M/month)
- **Concurrency limit** — max simultaneous in-flight requests (10 at once)

5.2 Algorithms

Fixed window counter Count requests per bucket (`user:123:2026-08-05T14:30`), reset each minute.

- ✓ Trivial: one Redis `INCR` + `EXPIRE`. Tiny memory footprint.
- ✗ **Boundary burst**: limit of 100/min allows 100 requests at 14:30:59 and 100 more at 14:31:00 — 200 in one second.

Sliding window log Store a timestamp per request in a sorted set; on each request drop entries older than the window and count what's left.

- ✓ Perfectly accurate, no boundary problem
- ✗ Memory grows with request count — 10k req/min per user means 10k stored timestamps. Expensive at scale.

Sliding window counter (*the usual compromise*) Keep the current and previous fixed-window counts and interpolate:

`estimate = current_count + previous_count × (overlap_fraction_of_previous_window)`

At 14:30:15 with a 1-min window: `count(14:30) + count(14:29) × 0.75`.

- ✓ Near-accurate, O(1) memory, cheap. **This is what Cloudflare uses.**
- ✗ Slightly approximate under very bursty traffic

Token bucket Bucket holds up to `B` tokens, refills at `r` tokens/sec. Each request consumes one; empty bucket → reject.

- ✓ **Allows bursts** up to `B` while enforcing average rate `r` — matches real user behaviour (idle, then a flurry of clicks)
- ✓ Only two values stored: `tokens` and `last_refill_ts`
- **This is usually the best default for user-facing APIs.**

Leaky bucket Requests queue and drain at a constant rate; overflow is dropped.

- ✓ Smooths output to a perfectly constant rate — good for protecting a fragile downstream (an SMS gateway with a hard 10/sec cap)
- ✗ Adds latency (queueing); no burst tolerance

5.3 Where and by what key

Layers — apply at more than one:

- **CDN/WAF** (Cloudflare) — stops volumetric attacks before they cost you anything
- **Reverse proxy** (nginx `limit_req_zone`) — cheap, protects PHP-FPM workers from saturation
- **Application** — the only layer that knows about users, plans, and endpoint cost

Keys — different keys for different threats:

- **IP** — the only option for anonymous traffic, but NAT/mobile carriers share IPs across thousands of users. Never rate-limit *only* by IP for logged-in users.
- **User ID / API key** — the right key for authenticated traffic
- **Endpoint-specific** — `/login` and `/otp/send` deserve far tighter limits (5/min) than `/buses` (300/min)
- **Composite** — `user + endpoint`, plus a global per-user cap
- **Cost-weighted** — a search costs 10 tokens, a health check costs 1

Behind a proxy, read the client IP from `X-Forwarded-For` **and** configure trusted proxies. Otherwise everyone shares your load balancer's IP, or an attacker spoofs the header.

5.4 The response

```
HTTP/1.1 429 Too Many Requests
Retry-After: 37
RateLimit-Limit: 100
RateLimit-Remaining: 0
RateLimit-Reset: 37
Content-Type: application/problem+json
```

```
{
  "type": "https://api.example.com/errors/rate-limit",
  "title": "Too many requests",
  "status": 429,
  "detail": "Limit of 100 requests per minute exceeded. Retry in 37 seconds."
}
```

- Send `RateLimit-*` headers on **every** response, not just 429s — that's how well-behaved clients self-throttle before hitting the wall.
- `Retry-After` in seconds (or as an HTTP date). Clients should honour it and add **jitter**; synchronised retries cause thundering herds.
- Older `X-RateLimit-*` names are still widespread; the IETF draft standardises the unprefixed `RateLimit-*` form.

5.5 Distributed implementation

Rate limiting only works if all your app servers share state. Redis is the standard answer.

Naive fixed window:

```
INCR ratelimit:user:123:1754400600
EXPIRE ratelimit:user:123:1754400600 60 # only on first increment
```

That's two round trips and a race between them. Use a **Lua script** so check-and-increment is atomic:

```
-- token bucket, atomic
local key    = KEYS[1]
local rate   = tonumber(ARGV[1]) -- tokens per second
local capacity = tonumber(ARGV[2])
local now    = tonumber(ARGV[3])

local b      = redis.call('HMGET', key, 'tokens', 'ts')
local tokens = tonumber(b[1]) or capacity
local last   = tonumber(b[2]) or now

tokens = math.min(capacity, tokens + (now - last) * rate)

local allowed = tokens >= 1
if allowed then tokens = tokens - 1 end

redis.call('HMSET', key, 'tokens', tokens, 'ts', now)
redis.call('EXPIRE', key, math.ceil(capacity / rate) * 2)

return { allowed and 1 or 0, tokens }
```

Fail-open vs fail-closed: if Redis is down, do you allow all traffic or reject it? For a public API, usually **fail open** — a rate limiter outage shouldn't become a full outage. For `/login` or payment endpoints, **fail closed**. Decide deliberately and write it down.

5.6 Laravel

```
// AppServiceProvider / RouteServiceProvider
RateLimiter::for('api', function (Request $request) {
    return $request->user()
        ? Limit::perMinute($request->user()->plan_limit)->by($request->user()->id)
        : Limit::perMinute(30)->by($request->ip());
});

RateLimiter::for('login', fn (Request $r) => [
    Limit::perMinute(5)->by($r->ip()),
    Limit::perMinute(5)->by($r->input('email')), // stops distributed attacks on one account
]);

Route::middleware('throttle:api')->group(* ... *);
Route::post('/login', ...) -> middleware('throttle:login');
```

Laravel's default is a **fixed window** backed by the cache store — set that store to Redis in production, not `file` or `array`. Returning an array of limits applies all of them.

Practice: implement the Lua token bucket above as a custom middleware, expose `RateLimit-*` headers, then load-test it with `ab` or `k6` and confirm you get exactly the expected number of 200s and 429s.

6. GraphQL Basics

6.1 The problem it solves

A mobile booking screen needs: trip details, operator name, and the first 3 seats. With REST:

```
GET /trips/8821          → 40 fields, you need 5      (over-fetching)
GET /trips/8821/operator → another round trip        (under-fetching)
GET /trips/8821/seats   → a third
```

Over-fetching wastes bandwidth on 3G. Under-fetching costs latency. The classic REST fix is a bespoke `/mobile/trip-screen` endpoint — which works, until you have 30 screens and the backend team becomes the bottleneck for every UI tweak.

GraphQL inverts control: **the client specifies the shape of the response.**

```
query {
  trip(id: "8821") {
    departsAt
    fare
    operator { name rating }
    seats(first: 3) { seatNo status }
  }
}
```

One request, exactly those fields.

6.2 The schema

The schema is the contract, written in SDL. It's strongly typed and introspectable — which is why GraphQL tooling (auto-complete, generated TS types, GraphQLQL) is so good.

```
scalar DateTime

enum SeatStatus { AVAILABLE HELD BOOKED }

type Operator {
  id: ID!
  name: String!
  rating: Float
  trips(first: Int = 10): [Trip!]!
}

type Seat {
  id: ID!
  seatNo: String!
  status: SeatStatus!
}

type Trip {
  id: ID!
  departsAt: DateTime!
  fare: Int!
  operator: Operator!
  seats(status: SeatStatus): [Seat!]!
}

input CreateBookingInput {
  tripId: ID!
  seatNos: [String!]!
  passengerName: String!
}

type Query {
  trip(id: ID!): Trip
  trips(operatorId: ID, first: Int = 20, after: String): [Trip!]!
}

type Mutation {
  createBooking(input: CreateBookingInput!): Booking!
  cancelBooking(id: ID!): Booking!
}

type Subscription {
  seatStatusChanged(tripId: ID!): Seat!
}
```

Type system pieces:

- **Scalars** — `Int`, `Float`, `String`, `Boolean`, `ID`, plus custom ones (`DateTime`, `Money`)
- `!` = non-nullable. `[Seat!]!` = a non-null list of non-null Seats.
- **Object types** — your entities
- **Input types** — separate from output types; mutation arguments must be inputs
- **Enums, Interfaces, Unions** — `union SearchResult = Trip | Operator`
- **Three root types** — `Query` (read), `Mutation` (write, executed serially), `Subscription` (real-time over WebSocket)

6.3 Resolvers

Every field has a resolver — a function returning that field’s value. Default resolvers just read the property off the parent, so you only write the interesting ones.

Signature: `resolve(parent, args, context, info)`

- **parent** — the value the parent field resolved to
- **args** — the field’s arguments
- **context** — per-request shared state: authenticated user, DB connection, loaders
- **info** — the AST of the query; useful for optimising (e.g. deciding what to eager-load)

Execution walks the tree: `Query.trip` → then `Trip.operator` with the Trip as `parent` → and so on. Sibling fields resolve in parallel.

6.4 N+1 — the defining GraphQL problem

```
{ trips(first: 50) { operator { name } } }
```

Naively: 1 query for trips + 50 queries for operators. And because the client controls query shape, you can’t just eager-load “the right things” ahead of time.

Solution: DataLoader. Batch and de-duplicate within a single request tick.

1. `Trip.operator` resolvers don’t query — they call `loader.load(operatorId)`
2. All 50 loads are collected during the tick
3. One batch function runs `SELECT * FROM operators WHERE id IN (...)`
4. Results are scattered back to the waiting resolvers

51 queries → 2. Per-request caching also de-dupes repeated IDs. **A GraphQL API without DataLoader (or equivalent) will fall over.**

6.5 Transport, errors, caching

- **One endpoint**, `POST /graphql`. (GET with a query string is possible and enables CDN caching, but is uncommon.)
- **Errors return HTTP 200** with an `errors` array. This is genuinely surprising and breaks monitoring built around status codes.
- **Partial success is normal:**

```
{
  "data": { "trip": { "departsAt": "...", "operator": null } },
  "errors": [{
    "message": "Operator service unavailable",
    "path": ["trip", "operator"],
    "extensions": { "code": "UPSTREAM_ERROR" }
  }]
}
```

Put machine-readable codes in `extensions`.

- **HTTP caching is largely lost** — every request is a POST to the same URL with a different body. You compensate with client-side normalised caches (Apollo, urql) keyed by `__typename` + `id`, and server-side caching at the resolver/DataLoader layer.
- **Persisted queries** — client sends a hash instead of the query text. Smaller payloads, GET-able (so CDN-cacheable), and lets you whitelist exactly which queries production accepts.

6.6 Pagination — Relay connections

The community standard wraps lists in a connection with cursors:

```
{
  trips(first: 20, after: "eyJ...") {
    edges {
      cursor
      node { id departsAt fare }
    }
    pageInfo { hasNextPage endCursor }
  }
}
```

`edges` exist so you can attach **edge metadata** — properties of the *relationship* rather than the node (e.g. `role` on a `user → organization` edge). Verbose, but standard, and every client library understands it.

6.7 Security — where GraphQL gets dangerous

Client-controlled query shape is also client-controlled server load.

- **Depth limiting** — `{ trip { operator { trips { operator { trips { ... } } } } } }` recurses forever. Cap depth at ~10.
- **Query complexity analysis** — assign each field a cost, multiply by list sizes, reject over a budget. `trips(first: 1000) { seats(first: 100) }` is 100,000 nodes from a tiny query string.
- **Amount limits** — require `first/last` on every list and cap it.
- **Disable introspection in production** — otherwise your entire schema, including internal fields, is a free map for an attacker.
- **Field-level authorisation** — REST authorises per endpoint; GraphQL must authorise per field, in the resolver. `Trip.internalMargin` needs its own check, because there's no endpoint to guard.
- **Rate limiting must be cost-based**, not request-based. Every query is one POST to one URL; counting requests measures nothing.
- **Batching abuse** — aliases let one request contain 1,000 operations: `a: trip(id:1) b: trip(id:2) ...`. Cap aliases/operations too.

6.8 REST vs GraphQL

	REST	GraphQL
Endpoints	Many	One
Response shape	Server decides	Client decides
Over/under-fetching	Common	Solved
HTTP caching	Native, excellent	Weak; needs client cache
Status codes	Full semantics	Mostly 200
Learning curve	Low	Moderate
Tooling / type safety	OpenAPI (opt-in)	Built into the schema
File uploads	Native	Needs a spec extension
Rate limiting	Per request	Must be cost-based
N+1 risk	Controlled by server	High; needs DataLoader
Best for	Public APIs, CRUD, caching-heavy, simple contracts	Many diverse clients, deeply nested data, fast-moving frontends, BFF layers

They aren't mutually exclusive. A very common production setup: REST between internal services (simple, cacheable, observable), GraphQL as a BFF for web and mobile clients.

In Laravel: `nuwave/lighthouse` — schema-first, SDL with directives (`@find`, `@paginate`, `@belongsTo`) that map straight onto Eloquent, with batch loading built in.

Practice Plan

Build one API — a small bus-booking service — and layer these on in order.

1. **REST foundation** — `buses`, `trips`, `bookings` with proper verbs, status codes, API Resources, FormRequest validation, and a consistent error format. Write down your envelope decision and stick to it.
2. **Pagination** — implement offset pagination, then hand-rolled cursor pagination on a 1M-row seeded table. Benchmark both at page 1 / 100 / 20,000 and record the numbers. This is the exercise that makes the concept stick.
3. **Filtering & sorting** — add whitelisted `filter[]`, `sort`, `fields`, `include`. Return 422 on unknown keys. Run `EXPLAIN` on your three most common filter combinations and add the indexes they need.
4. **Versioning** — ship a v2 of one endpoint with a renamed field and a flattened nested object. Keep v1 alive against the same service layer. Add `Deprecation` and `Sunset` headers to v1.
5. **Rate limiting** — Laravel's `throttle` first, then replace it with the Redis Lua token bucket. Expose `RateLimit-*` headers on every response. Load-test and verify the 200/429 split matches your maths.
6. **GraphQL** — install Lighthouse, expose `trips` and `bookings`, deliberately create the N+1, observe it in Telescope/Clockwork, then fix it with batch loading. Add depth and complexity limits.

Self-check questions:

- What's the difference between 401 and 403? 400 and 422?
- Which HTTP methods are idempotent, and why does it matter for retry logic?
- Why does offset pagination get slower with depth, and why does cursor pagination not?
- Why does `created_at` alone make a broken cursor?
- Name two attacks a filter whitelist prevents.
- What makes a change breaking? Give three examples of non-breaking changes.
- Compare token bucket and sliding window counter. When would you choose each?
- Should a rate limiter fail open or fail closed? What does it depend on?
- Why does GraphQL return 200 for errors, and what does that break?
- Why doesn't request-count rate limiting work for GraphQL?